`ActorRef[Res] => Req` and not an object. So the question is this: How is the `Worker.Parse` class used as a function? You saw a hint in the previous section, but let's be more explicit.

If you're not familiar with Scala, this is probably confusing. Classes and objects can be treated like functions if they have an `apply` method and a `case` class like the way `Worker.Parse` has the `apply` method under the hood:

```
object Worker {
  ...
  final case class Parse(replyTo: ActorRef[Worker.Response])
      extends Command
  ...
}
```

Thanks to the compiler, the `case` class `Worker.Parse(replyTo: ActorRef[Worker.Response])` creates a method like the following:

```
def apply(replyTo: ActorRef[Worker.Response]): Worker.Parse =
➥new Worker.Parse(replyTo)
```

Because this method is available, the compiler understands that this is what you mean when you put `Worker.Parse` in the `ask` method. The compiler concludes that you are referring to the `apply` function of this `case` class because that is the only thing that makes sense in the context of the `createRequest` signature.

And that's it: thanks to syntactic sugar and cleverness of the compiler, you pass `Worker.Parse.apply()` to this method using